The Data Link Layer

Design Issues, Framing, Error Control, and Protocols

Dr Vijay Ukani

Based on Computer Networks by Andrew Tanenbaum

Disclaimer

- The contents presented in the slides is compilation of information from various sources. The source of information is mentioned on the slides.
- The slides are used for academic purpose. The distribution and use of the slides for academic use is permitted.

Data Link Layer Introduction

Core Objectives

- Algorithms are designed for achieving **reliable, efficient communication** of whole units.
- These units are called **frames** (as opposed to raw bits handled by the Physical Layer).
- Communication occurs between two machines connected by a channel that conceptually acts like a wire (e.g., telephone line, coaxial cable, wireless).
- The essential property of this "wire-like" channel is that bits are delivered in exactly the same order they were sent.

The Challenge

For an ideal channel (no distortion, unlimited bandwidth, and no delay), the job of the data link layer would be trivial.
However, the real world presents challenges:

- Limited bandwidth
- Signal distortions and errors
- Transmission delays

These factors make the Data Link Layer's job complex and vital.

Data Link Layer Design Issues

The Physical layer delivers raw bits of information to and from the data link layer. Thus, the primary functions of the Data Link Layer are:

- **Service Interface:** Providing a well-defined service interface to the network layer above it.
- **Framing:** Taking packets from the Network layer and encapsulating them into frames.
- **Error Handling:** Dealing with transmission errors caused by the physical medium.
- **Flow Control:** Regulating the flow of data so that slow receivers are not swamped by fast senders.

Frame Management is the core of what the Data Link Layer does.

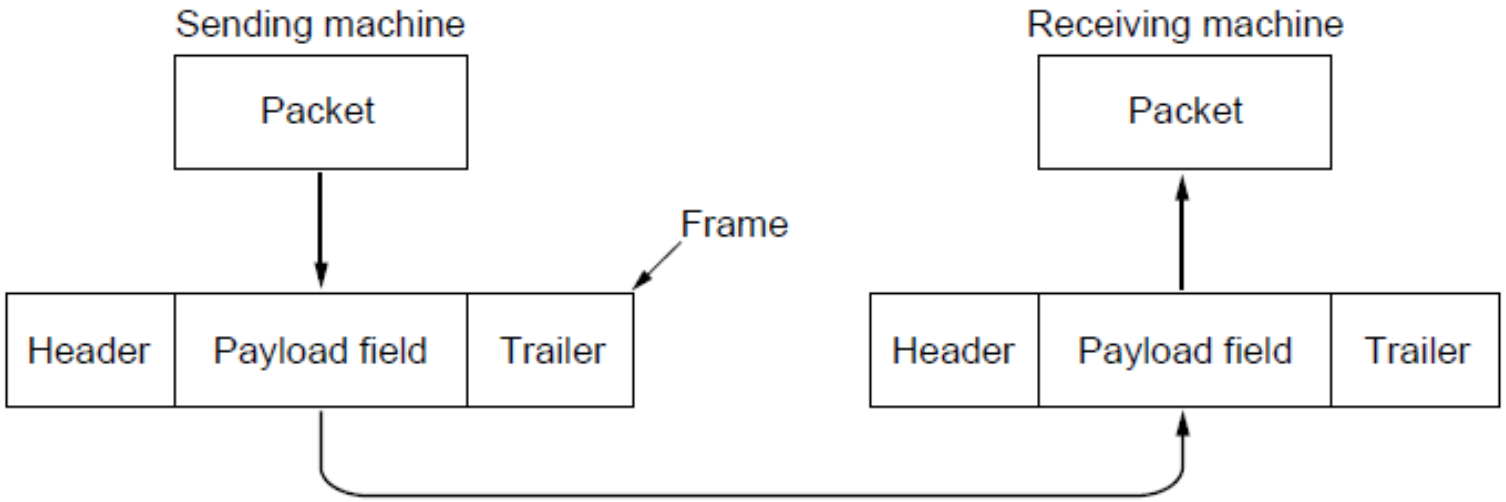
Packets and Frames

Relationship

- The Network Layer generates data units called **Packets**.
- The Data Link Layer encapsulates these packets into **Frames** for transmission.

Each frame consists of three parts:

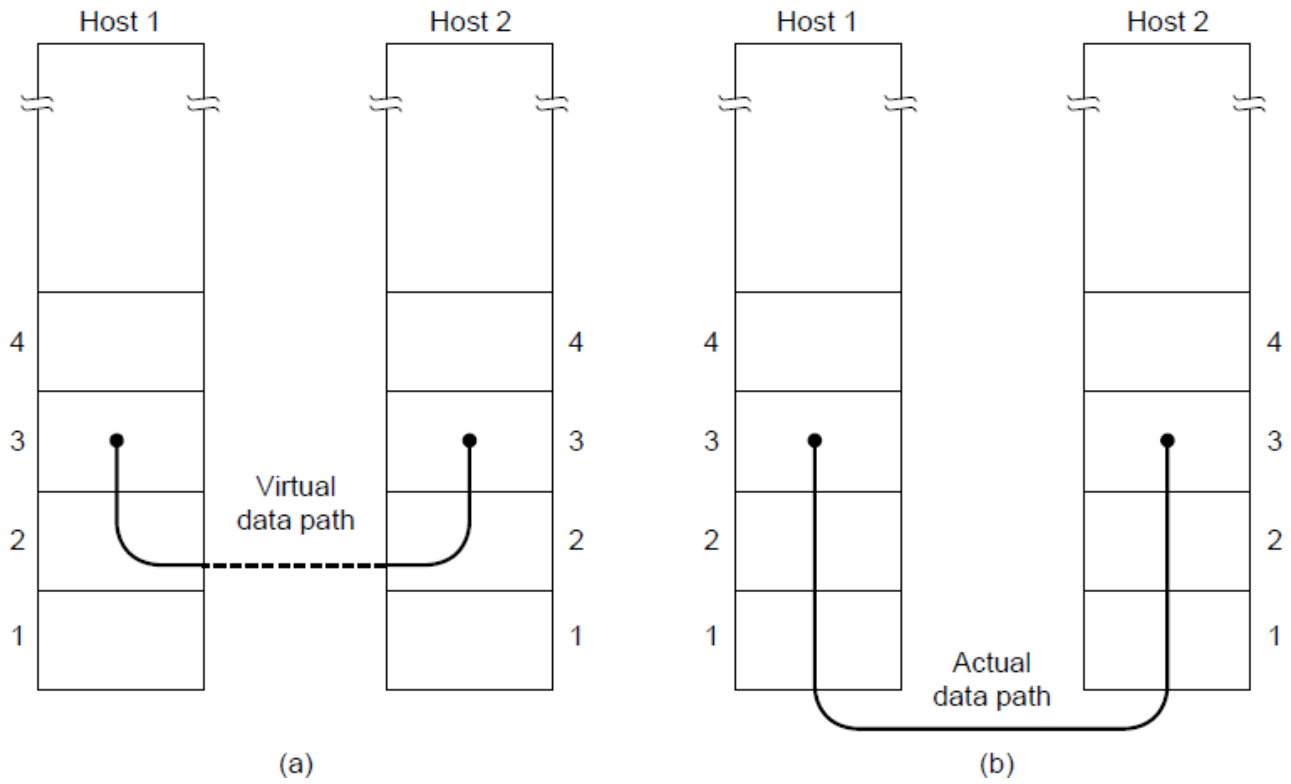
- **Frame Header:** Contains control information (addresses, sequence numbers).
- **Payload Field:** Holds the packet data passed down from the network layer.
- **Frame Trailer:** Typically contains error detection information (checksum).



Services Provided to the Network Layer

Primary Function

- The primary service function of the data link layer is to transfer data from the network layer on the source machine to the network layer on the destination machine.
- A process in the network layer hands some bits (a packet) to the data link layer for transmission.
- The job of the data link layer is to transmit the packet to the destination machine so it can be handed over to the network layer process there.



(a) Virtual communication. (b) Actual communication.

Possible Services Offered

The Data Link Layer can offer different types of services to the Network Layer, depending on the requirements of the system. The three main categories are:



Unacknowledged Connectionless Service



Acknowledged Connectionless Service



Acknowledged Connection-Oriented Service

Unacknowledged Connectionless Service

- It consists of having the source machine send independent frames to the destination machine without having the destination machine acknowledge them.
- No logical connection is established beforehand or released afterward.
- If a frame is lost due to noise on the line, no attempt is made to detect the loss or recover from it in the data link layer.
- **Use Cases:** Appropriate when the error rate is very low (recovery left to higher layers) or for real-time traffic where late data is worse than bad data.
- **Examples:** Most LANs use this (e.g., standard Ethernet), Voice over IP (VoIP).

Acknowledged Connectionless Service

- Still no logical connection used, but each frame sent is individually acknowledged.
- The sender knows whether a specific frame has arrived safely or was lost.
- Typically, the protocol uses a specific time period. If the timeout passes without getting an acknowledgment, it will re-send the frame.
- **Use Cases:** Useful over unreliable channels where the probability of losing a frame is high.
- **Examples:** Wireless systems like 802.11 (WiFi). It's better to fix errors locally in the data link layer than force the transport layer to retransmit an entire large message.

Acknowledged Connection-Oriented Service

- The most sophisticated service. Source and destination establish a connection before any data is transferred.
- Each frame sent is numbered.
- The data link layer guarantees that each frame sent is indeed received, and received exactly once.
- Guarantees that all frames are received in the correct order.
- **Three Distinct Phases:**
    - **Connection Establishment:** Initialize variables and counters.
    - **Data Transmission:** One or more frames are transmitted.
    - **Connection Release:** Freeing up variables, buffers, and resources.
- **Examples:** Long-distance point-to-point satellite channel communication.

Framing: The Need

- To provide service to the network layer, the data link layer must use the service provided to it by the physical layer.
- The stream of data bits provided by the physical layer is **not guaranteed to be error-free**.
- **Types of Errors:**
  - Number of received bits does not match the number of transmitted bits (deletion or insertion of bits).
  - Alteration of bit values (a 0 becomes a 1, or vice versa).
- It is up to the data link layer to detect and, if necessary, correct these errors.

Framing: The Mechanism

Transmission in the data link layer requires breaking the continuous bit stream into manageable blocks.



1. Break Stream

Break up the bit stream into discrete, identifiable units called frames.



2. Compute Checksum

Compute a checksum (a short mathematical summary) for the payload of each frame.



3. Transmit

Include the checksum in the frame's trailer before transmitting it over the medium.



4. Verify

Receiver computes its own checksum and compares. A mismatch indicates an error.

Note: Finding the start and end of frames (synchronization) is more difficult than one might think!

Framing Methods

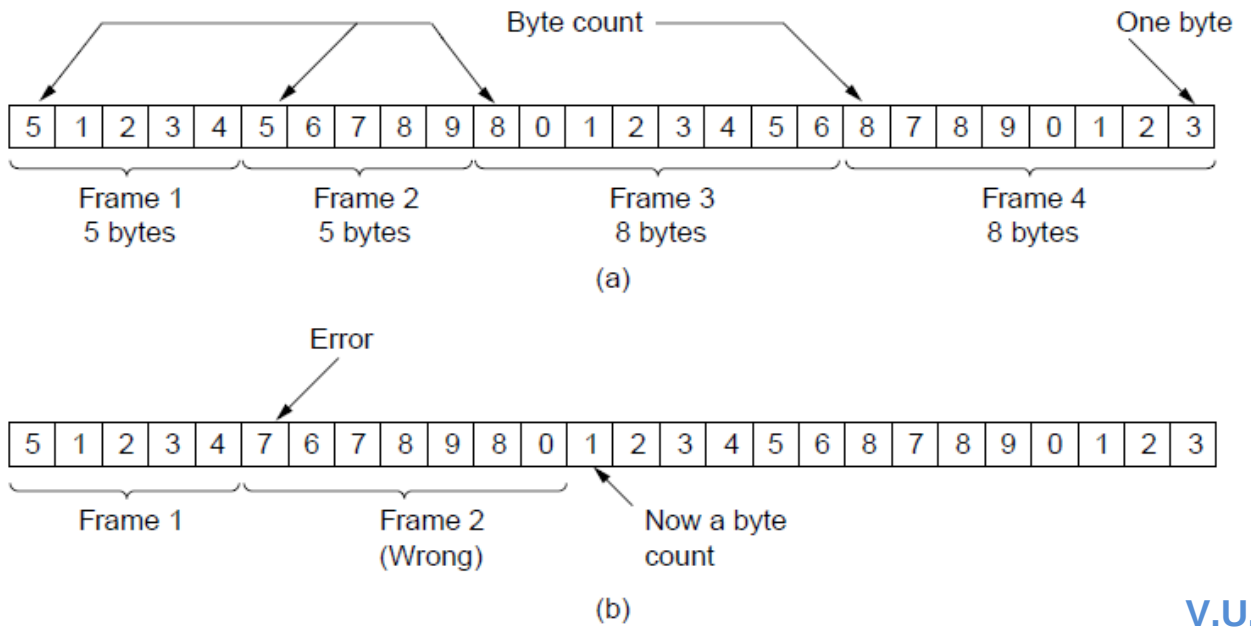
There are four common methods used to delimit frames in the bit stream:

- **Byte Count:** Uses a field in the header to specify the frame length.
- **Flag Bytes with Byte Stuffing:** Uses special byte markers and escape characters.
- **Flag Bits with Bit Stuffing:** Uses special bit patterns and inserts bits to prevent confusion.
- **Physical Layer Coding Violations:** Uses invalid signals at the physical layer to mark boundaries.

Framing Method 1: Byte Count

- This method uses a field in the header to specify the number of bytes in the frame.
- Once the header information is received, the receiver uses this count to determine where the end of the frame is located.
- **The Trouble:** If the count field is incorrectly received due to a transmission error, the destination gets out of sync.
- The destination may detect an error (e.g., bad checksum), but it has no way to locate the start of the next frame to resynchronize.

Frame 1 (5 bytes)	Frame 2 (5 bytes)	Frame 3 (8 bytes)
5 1 2 3 4	5 6 7 8 9	8 0 1 2 3 4 5 6



Framing Method 2: Flag Bytes with Byte Stuffing

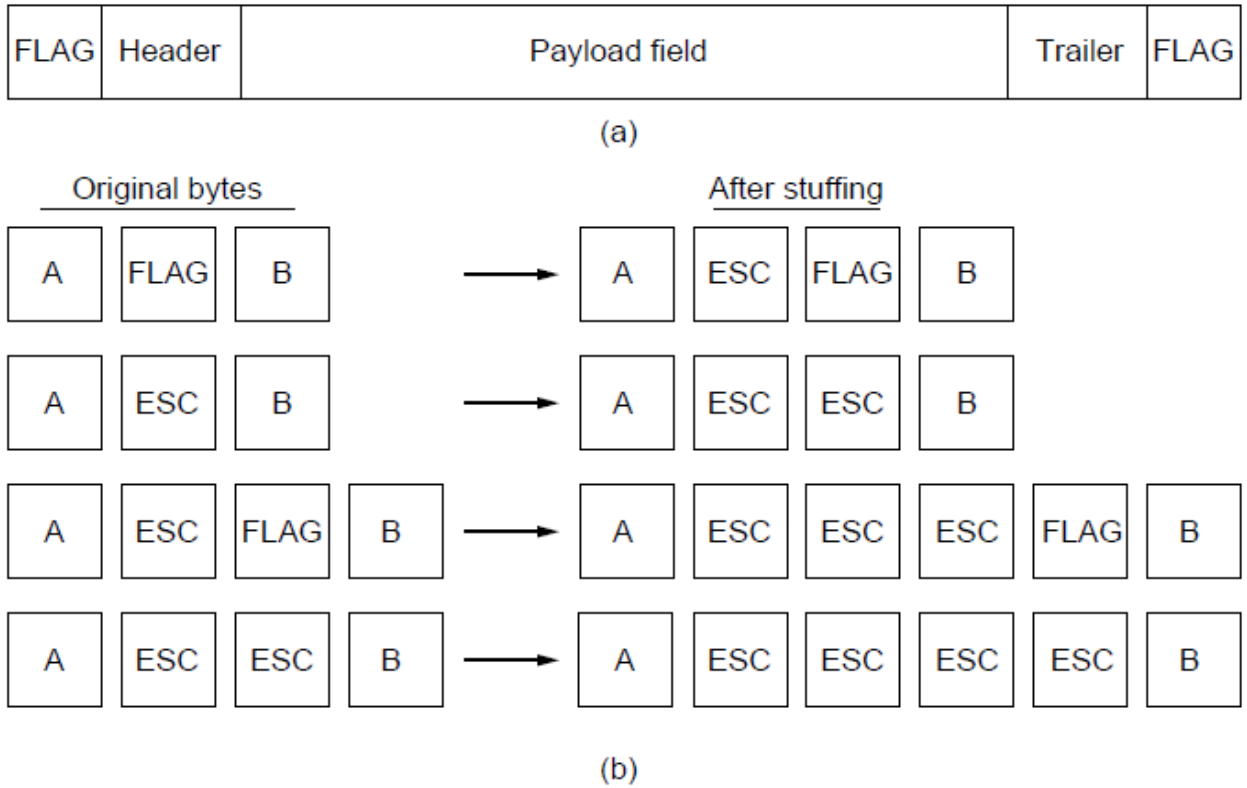
- This method solves the resynchronization problem by having each frame start and end with a special byte, called a **FLAG** byte.
- If synchronization is lost, the receiver just searches for the next FLAG byte.
- **The Problem:** What if the actual payload data contains a byte identical to the FLAG byte (e.g., binary data, images)?
- **The Solution (Byte Stuffing):** The sender inserts a special **Escape (ESC)** character just before any accidental FLAG byte in the data. The receiver removes the ESC before passing data to the network layer.

Byte Stuffing Examples

Four examples of byte sequences before and after byte stuffing:

Scenario	Original Bytes (Payload)	After Stuffing (Transmitted)
Data contains FLAG	A | **FLAG** | B	A | **ESC** | **FLAG** | B
Data contains ESC	A | **ESC** | B	A | **ESC** | **ESC** | B
Data contains ESC then FLAG	A | **ESC** | **FLAG** | B	A | **ESC** | **ESC** | **ESC** | **FLAG** | B
Data contains two ESCs	A | **ESC** | **ESC** | B	A | **ESC** | **ESC** | **ESC** | **ESC** | B

Byte Stuffing Examples



a) A frame delimited by flag bytes.

b) Four examples of byte sequences before and after byte stuffing.

Framing Method 3: Flag Bits with Bit Stuffing

- Achieves framing at the bit level rather than the byte level.
- Developed for High-level Data Link Control (HDLC) protocol and used in USB.
- Each frame begins and ends with a specific bit pattern: 01111110 (0x7E), which acts as the Flag.
- **Bit Stuffing Rule:** Whenever the sender's data link layer encounters five consecutive '1's in the data stream, it automatically stuffs a '0' bit into the outgoing stream.
- The receiver destuffs the '0' upon seeing five '1's followed by a '0'.

(a) Original Data: 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 0

(b) Stuffed Data: 0 1 1 0 1 1 1 1 1 11111 11111 10010

^ ^ ^ 0 0 0

Stuffed Stuffed Stuffed

bit bit bit

(c) Destuffed Data: 0 1 1 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 1 0

Framing Method 4: Physical Layer Coding Violations

- Many networks use specific encoding techniques at the Physical Layer, such as Manchester or Differential Manchester encoding.
- These techniques require a signal transition in the middle of every bit transmission period (e.g., high-to-low or low-to-high).
- **The Method:** The Data Link layer can deliberately violate this rule by using a signal without a transition (e.g., a high-high or low-low pair) to indicate the beginning or end of a frame.
- Since these violations cannot occur in valid data encoding, they unambiguously mark frame boundaries without needing escape characters or stuffing.

Error Control

- After identifying frame boundaries, the data link layer must handle transmission errors.
- **Goal:** Ensuring that all frames are delivered to the network layer at the destination in proper order.
- For *unacknowledged connectionless service*, it is OK for the sender to output frames regardless of reception.
- For *reliable connection-oriented service*, it is NOT OK to ignore errors.
- **Mechanism:**
  - Receiver sends back special control frames (Acknowledgements - ACKs or Negative ACKs - NAKs) to provide feedback.
  - **Timers and Sequence Numbers** are necessary for the sender to handle cases where a frame or the acknowledgement is completely lost.

Flow Control

An important design issue when a fast sender is communicating with a slow receiver. The sender must be throttled to prevent buffer overflow at the receiver.

Feedback-based Flow Control

The receiver explicitly sends back information giving the sender permission to send more data, or telling it how much buffer space is left. Most DLL protocols use this approach.

Rate-based Flow Control

A built-in mechanism limits the transmission rate based on a negotiated contract, without needing continuous feedback. (e.g., ABR service in ATM).

Credit-based Flow Control

The sender is given explicit 'credit' (e.g., number of bytes). It transmits until credits are exhausted, then pauses until more credits arrive. (Used in TCP).

Error Detection and Correction

Because physical channels are imperfect, we must reliably transmit groups of bits. There are two basic strategies involving adding redundancy:



Error-Correcting Codes

Include enough redundant information to enable the receiver to deduce exactly what the transmitted data must have been.
Used heavily on highly noisy channels (like wireless) where retransmission is expensive.



Error-Detecting Codes

Include just enough redundancy to allow the receiver to deduce that an error has occurred (but not which specific bit flipped).
Requires retransmission. Used heavily on reliable channels (like fiber optics).

Error Codes Fundamentals

- Error codes are examined in the Data Link Layer because it's the first place we encounter the problem of reliably transmitting groups of bits.
- All codes work by adding redundancy to the information.
- A frame consists of **m** data bits (message) and **r** redundant bits (check bits).
- **n** is the total length of the block: **n = m + r**.
- This is referred to as an **(n, m) code**.
- The **code rate** is m/n. (Ranges from 1/2 for noisy channels to close to 1 for high-quality channels).

Hamming Distance

- The number of bit positions in which two codewords differ is called the **Hamming Distance**.
- If two codes are *d* distance apart, it requires *d* single-bit errors to convert one into the other.

Example of calculating Hamming Distance:

Transmitted :	10001001
Received :	10110001
XOR :	00111000

The XOR operation gives 3 bits that are different. Therefore, the Hamming Distance = 3.

Hamming Distance Limits

- To reliably **detect** *d* errors, you need a distance ***d + 1*** code.
- To completely **correct** *d* errors, you need a distance ***2d + 1*** code.

Example (Distance 5 Code):

```
Valid codes:
0000000000
0000011111
1111100000
1111111111
```

Minimal Distance is 5. Therefore it can correct double (2) errors, or detect quadruple (4) errors.

- If 0000000111 is received, it's a double-bit error from 0000011111.
- However, you cannot perform double error correction and at the same time detect quadruple errors.

Lower Limit on Check Bits

- Error correction requires evaluation of candidate codewords, which can be time-consuming. Design minimizes this search time.
- To correct single-bit errors in an m-bit message using r check bits, we must have enough valid states to identify the correct message or any of the m+r single-bit errors.
- The theoretical requirement for the number of check bits r needed to correct a single error in a message of length m is:

$$(m + r + 1) \leq 2^{r}$$

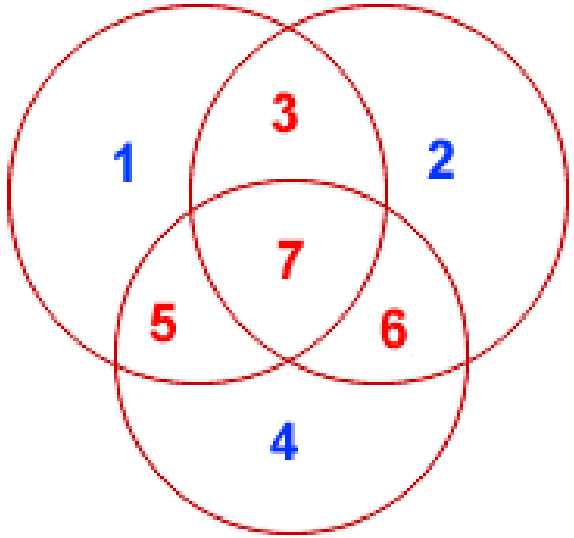
The Hamming Code: Construction

- A widely used error-correcting code capable of correcting single-bit errors.
- **Bit Positions:** Bits are numbered starting from 1 (left to right).
- **Check Bits:** Bit positions that are powers of 2 (p1, p2, p4, p8...) are used as Check bits.
- **Data Bits:** The remaining positions (m3, m5, m6, m7, m9...) are filled with m data bits.
- Example: A message with 4 data bits (D) requires 3 error-correcting parity bits (P), creating a 7-bit codeword. This is a **(7,4) code**.

Bit Position	**1**	**2**	**3**	**4**	**5**	**6**	**7**
Type	**P1**	**P2**	D1	**P4**	D2	D3	D4

The Hamming Code: Parity Intersections

The three parity bits (1, 2, 4) are related to the data bits (3, 5, 6, 7).



- Each overlapping circle corresponds to one parity bit and defines the four bits contributing to that parity computation.
- Each circle must have **EVEN parity**.
- Changing any one bit (1-7) uniquely affects the parity bits.
- Example: Data bit 3 contributes to Parity 1 and Parity 2.

The Hamming Code: Encoding Example

Let's send the 4-bit message: **1 1 0 1** (D4 D3 D2 D1)

We place the data bits in positions 7, 6, 5, and 3. We calculate P1, P2, P4 to ensure EVEN parity for each group.

Bit Position: 7 6 5 4 3 2 1
Data/Parity: 1 1 0 P4 1 P2 P1

P1 covers bits 1, 3, 5, 7 -> P1, 1, 0, 1 -> To be EVEN, P1 = 0
P2 covers bits 2, 3, 6, 7 -> P2, 1, 1, 1 -> To be EVEN, P2 = 1
P4 covers bits 4, 5, 6, 7 -> P4, 0, 1, 1 -> To be EVEN, P4 = 0

Final 7-bit Codeword: 1 1 0 0 1 1 0

The Hamming Code: Error Correction

Suppose the message 1 1 0 0 1 1 0 is sent, but a single bit error occurs in bit position 5, so 1 1 1 0 1 1 0 is received.

Received Codeword: 1 1 1 0 1 1 0 (Bit 5 flipped)

Bit Position: 7 6 5 4 3 2 1

Check Parities:

P1 (1,3,5,7): 0, 1, 1, 1 -> Parity is ODD (NOT OK! -> 1)

P2 (2,3,6,7): 1, 1, 1, 1 -> Parity is EVEN (OK! -> 0)

P4 (4,5,6,7): 0, 1, 1, 1 -> Parity is ODD (NOT OK! -> 1)

The bad parity bits labeled 1 0 1 (reading P4 P2 P1) point directly to the bad bit because 101 in binary equals **5**. We flip bit 5 to correct it!

Cost: 3 bits added to a 4-bit message to correct any single bit error.

Error-Detecting Codes

For highly reliable channels, detecting an error and asking for a retransmission is more efficient than the overhead of error-correcting codes.

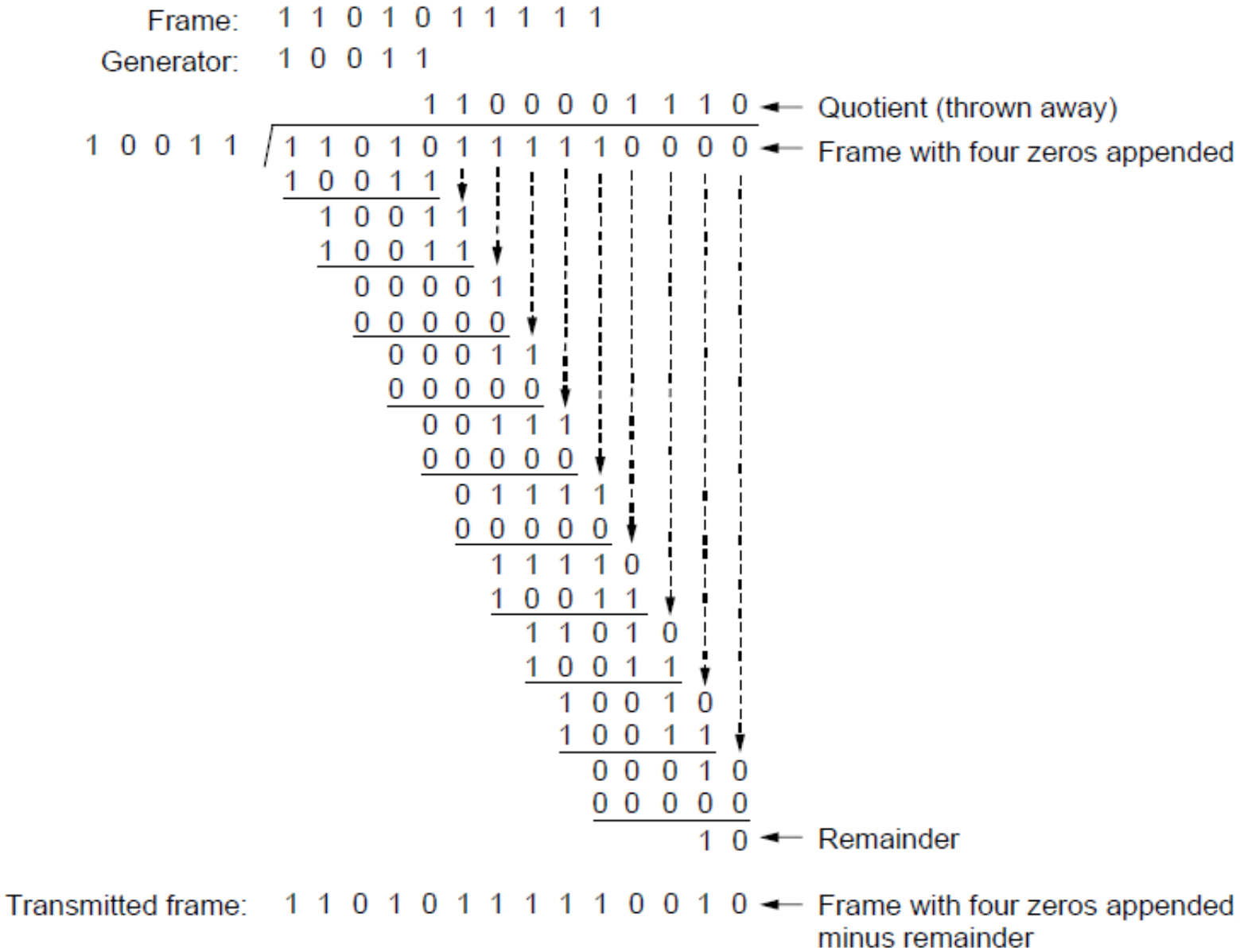
Cyclic Redundancy Check (CRC)

- Based on polynomial arithmetic over modulo 2.
- Highly robust against burst errors.
- Sender and receiver agree on a generator polynomial $G(x)$.
- Sender performs division, appends remainder to frame.
- Receiver performs division; if remainder is 0, frame is valid.

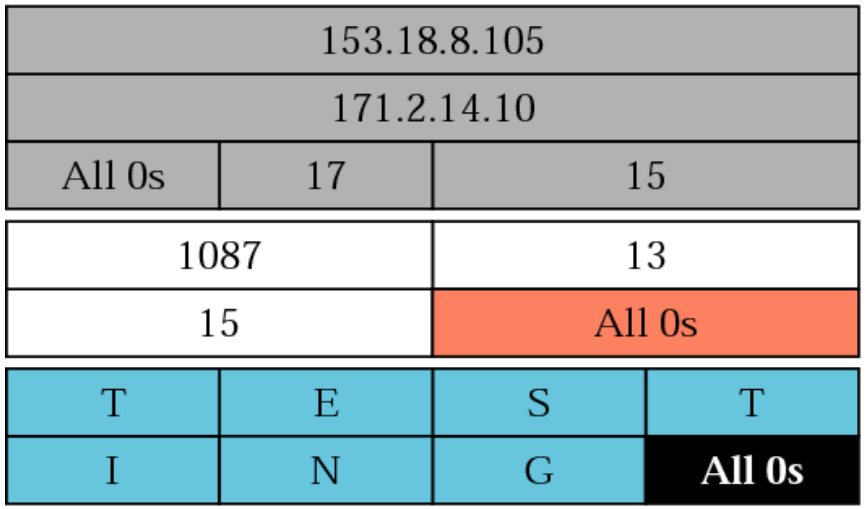
Internet Checksum

- Used extensively in TCP/IP (IPv4, TCP, UDP).
- Treats data as 16-bit integers.
- Adds them using 1's complement arithmetic.
- Result is bitwise inverted.
- Simpler in software than CRC, but slightly less robust.

Error-Detecting Codes – CRC Example



Error-Detecting Codes – Checksum Example

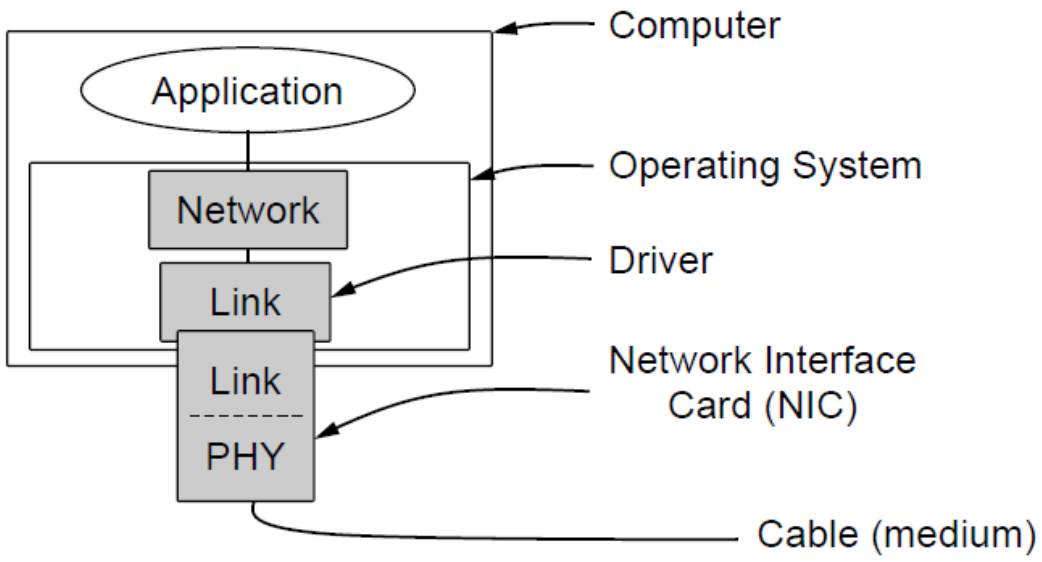


10011001 00010010	153.18
00001000 01101001	8.105
10101011 00000010	171.2
00001110 00001010	14.10
00000000 00010001	0 and 17
00000000 00001111	15
00000100 00111111	1087
00000000 00001101	13
00000000 00001111	15
00000000 00000000	0 (checksum)
01010100 01000101	T and E
01010011 01010100	S and T
01001001 01001110	I and N
01000111 00000000	G and 0 (padding)
10010110 11101011	Sum
01101001 00010100	Checksum

Example calculation (in UDP) of the Checksum

Implementation Architecture

- **The Physical (PHY) process** and time-critical parts of the data link layer run on dedicated hardware called a Network Interface Card (NIC).
- The rest of the link layer process and the network layer process run on the main CPU as part of the Operating System, usually taking the form of a device driver.



Protocol Setup & Key Assumptions

To analyze data link protocols, we make a few simplifying assumptions:

- Machine A wants to send a long stream of data to machine B, using a reliable, connection-oriented service.
- Machine A is assumed to have an infinite supply of data ready to send and never has to wait for data to be produced.
- Machines do not crash mid-transmission.

The protocols use predefined structures in C-like code (e.g., MAX_PKT, packet, frame containing kind, seq, ack, and info) and event-handling functions (wait_for_event, from_physical_layer, start_timer).

Protocol Setup

```
#define MAX_PKT 1024                                /* determines packet size in bytes */

typedef enum {false, true} boolean;                 /* boolean type */
typedef unsigned int seq_nr;                        /* sequence or ack numbers */
typedef struct {unsigned char data[MAX_PKT];} packet;  /* packet definition */
typedef enum {data, ack, nak} frame_kind;           /* frame_kind definition */


typedef struct {                                    /* frames are transported in this layer */
  frame_kind kind;                                  /* what kind of frame is it? */
  seq_nr seq;                                       /* sequence number */
  seq_nr ack;                                       /* acknowledgement number */
  packet info;                                      /* the network layer packet */
} frame;
```

```
/* Wait for an event to happen; return its type in event. */
void wait_for_event(event_type *event);

/* Fetch a packet from the network layer for transmission on the channel. */
void from_network_layer(packet *p);

/* Deliver information from an inbound frame to the network layer. */
void to_network_layer(packet *p);

/* Go get an inbound frame from the physical layer and copy it to r. */
void from_physical_layer(frame *r);

/* Pass the frame to the physical layer for transmission. */
void to_physical_layer(frame *s);

/* Start the clock running and enable the timeout event. */
void start_timer(seq_nr k);

/* Stop the clock and disable the timeout event. */
void stop_timer(seq_nr k);
```

Definitions needed in the protocols to follow. These definitions are located in the file protocol.h.

Protocol 1: Utopian Simplex Protocol

- A purely theoretical protocol acting as a baseline.
- Data is transmitted in **one direction only** (simplex).
- Assumes a perfect, **error-free** communication channel.
- Assumes the receiver has **infinite processing speed** and infinite buffer capacity.
- The sender constantly loops, pumping data out onto the line as fast as possible without waiting for any feedback.

Protocol 1: Utopian Simplex Protocol

```
typedef enum {frame_arrival} event_type;
#include "protocol.h"

void sender1(void)
{
  frame s;                          /* buffer for an outbound frame */
  packet buffer;                    /* buffer for an outbound packet */

  while (true) {
      from_network_layer(&buffer);  /* go get something to send */
      s.info = buffer;              /* copy it into s for transmission */
      to_physical_layer(&s);        /* send it on its way */
  }                                 /* Tomorrow, and tomorrow, and tomorrow,
                                       Creeps in this petty pace from day to day
                                       To the last syllable of recorded time.
                                           – Macbeth, V, v */
}

void receiver1(void)
{
  frame r;
  event_type event;                 /* filled in by wait, but not used here */

  while (true) {
      wait_for_event(&event);       /* only possibility is frame_arrival */
      from_physical_layer(&r);      /* go get the inbound frame */
      to_network_layer(&r.info);    /* pass the data to the network layer */
  }
}
```

Protocol 2: Simplex Stop-and-Wait

- Drops the unrealistic receiver assumptions of Protocol 1.
- The receiver now has **finite buffer capacity** and finite processing speed.
- Must prevent the sender from flooding the receiver.
- **Mechanism:** The sender transmits one frame and completely stops. It waits for a dummy "go-ahead" frame from the receiver before transmitting the next piece of data.

Protocol 2: Simplex Stop-and-Wait

```
/* Protocol 2 (Stop-and-wait) also provides for a one-directional flow of data from
   sender to receiver. The communication channel is once again assumed to be error
   free, as in protocol 1. However, this time the receiver has only a finite buffer
   capacity and a finite processing speed, so the protocol must explicitly prevent
   the sender from flooding the receiver with data faster than it can be handled. */

typedef enum {frame_arrival} event_type;
#include "protocol.h"

void sender2(void)
{
  frame s;                          /* buffer for an outbound frame */
  packet buffer;                    /* buffer for an outbound packet */
  event_type event;                 /* frame_arrival is the only possibility */

  while (true) {
      from_network_layer(&buffer);  /* go get something to send */
      s.info = buffer;              /* copy it into s for transmission */
      to_physical_layer(&s);        /* bye-bye little frame */
      wait_for_event(&event);       /* do not proceed until given the go ahead */
  }
}
```

Protocol 2: Simplex Stop-and-Wait

```
void receiver2(void)
{
  frame r, s;                          /* buffers for frames */
  event_type event;                    /* frame_arrival is the only possibility */
  while (true) {
        wait_for_event(&event);        /* only possibility is frame_arrival */
        from_physical_layer(&r);       /* go get the inbound frame */
        to_network_layer(&r.info);     /* pass the data to the network layer */
        to_physical_layer(&s);         /* send a dummy frame to awaken sender */
  }
}
```

Protocol 3: PAR for Noisy Channels

Positive Acknowledgement with Retransmission (PAR)

- Real-world channels lose and corrupt frames.
- If a frame is damaged, the receiver's hardware detects this via checksum and drops the frame.
- The sender uses a **Timer**. If an acknowledgement isn't received before timeout, it retransmits the frame.

Consider

- The network layer on A gives packet 1 to its data link layer. The packet is correctly received at B and passed to the network layer on B. B sends an acknowledgement frame back to A.
- The acknowledgement frame gets lost completely.
- The data link layer on A eventually times out and sends the frame containing packet 1 again
- The duplicate frame also arrives at the data link layer on B perfectly and sent to network layer.
- The protocol will fail. Some way for the receiver to be able to distinguish a frame that it is seeing for the first time from a retransmission is needed

- **The Problem:** What if the ACK is lost? The sender retransmits, and the receiver gets a duplicate frame, breaking the protocol.
- **The Solution:** The receiver needs to distinguish new frames from retransmissions. We add a **Sequence Number** to the frame header. A 1-bit sequence number (0 or 1) is sufficient for stop-and-wait.

Protocol 3: PAR for Noisy Channels

- Since a small frame header is desirable, the question arises: What is the minimum number of bits needed for the sequence number?
- The only ambiguity in this protocol is between a frame, m, and its direct successor, m + 1.
- If frame m is lost or damaged, the receiver will not acknowledge it, so the sender will keep trying to send it.
- Once it has been correctly received, the receiver will send an acknowledgement to the sender.
- Depending upon whether the acknowledgement frame gets back to the sender correctly or not, the sender may try to send m or m + 1.
- A 1-bit sequence number (0 or 1) is therefore sufficient

Protocol 3: PAR for Noisy Channels

```
#define MAX_SEQ 1                                   /* must be 1 for protocol 3 */
typedef enum {frame_arrival, cksum_err, timeout} event_type;
#include "protocol.h"

void sender3(void)
{
  seq_nr next_frame_to_send;                        /* seq number of next outgoing frame */
  frame s;                                          /* scratch variable */
  packet buffer;                                    /* buffer for an outbound packet */
  event_type event;
  next_frame_to_send = 0;                           /* initialize outbound sequence numbers */
  from_network_layer(&buffer);                      /* fetch first packet */
  while (true) {
      s.info = buffer;                              /* construct a frame for transmission */
      s.seq = next_frame_to_send;                   /* insert sequence number in frame */
      to_physical_layer(&s);                        /* send it on its way */
      start_timer(s.seq);                           /* if answer takes too long, time out */
      wait_for_event(&event);                       /* frame_arrival, cksum_err, timeout */
      if (event == frame_arrival) {
          from_physical_layer(&s);                  /* get the acknowledgement */
          if (s.ack == next_frame_to_send) {
              stop_timer(s.ack);                    /* turn the timer off */
              from_network_layer(&buffer);          /* get the next one to send */
              inc(next_frame_to_send);              /* invert next_frame_to_send */
          }
      }
  }
}
```

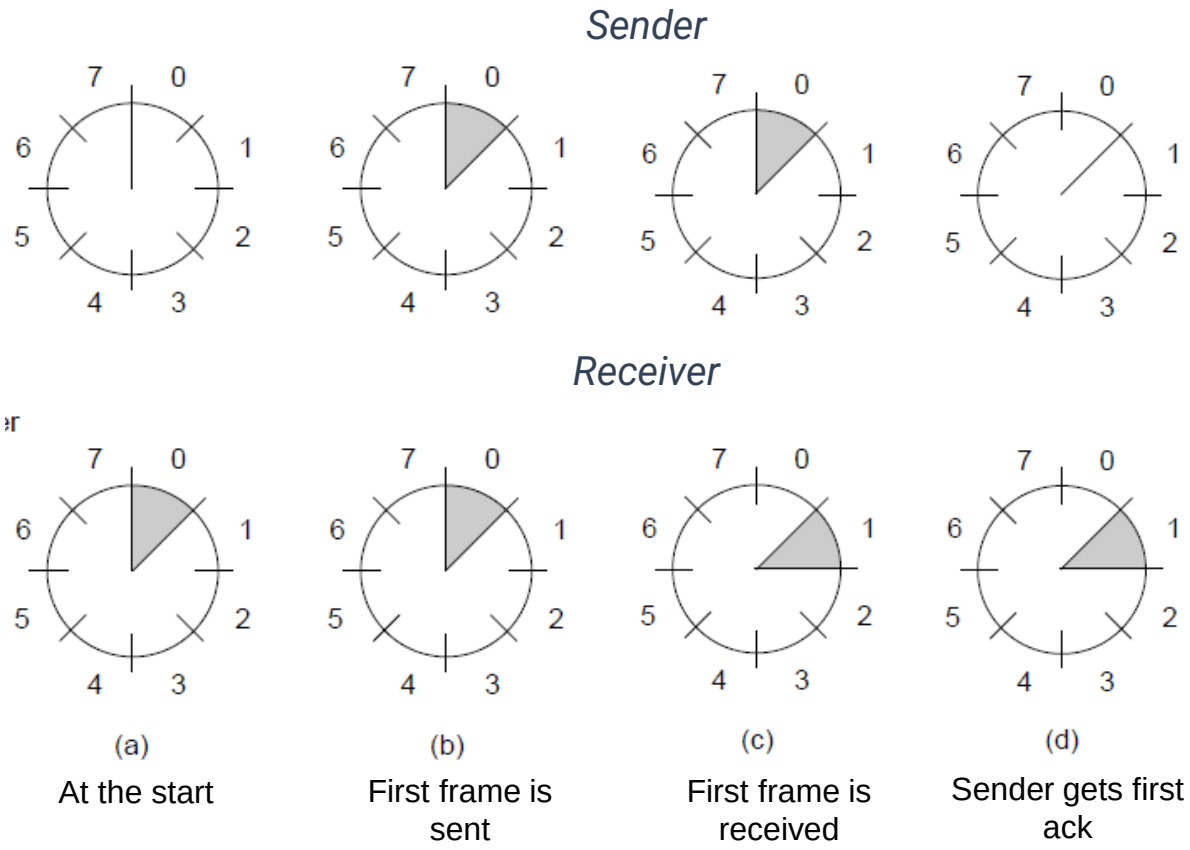
Protocol 3: PAR for Noisy Channels

```
void receiver3(void)
{
  seq_nr frame_expected;
  frame r, s;
  event_type event;

  frame_expected = 0;
  while (true) {
       wait_for_event(&event);                  /* possibilities: frame_arrival, cksum_err */
       if (event == frame_arrival) {            /* a valid frame has arrived */
            from_physical_layer(&r);            /* go get the newly arrived frame */
            if (r.seq == frame_expected) {      /* this is what we have been waiting for */
                 to_network_layer(&r.info);     /* pass the data to the network layer */
                 inc(frame_expected);           /* next time expect the other sequence nr */
            }
            s.ack = 1 – frame_expected;         /* tell which frame is being acked */
            to_physical_layer(&s);              /* send acknowledgement */
       }
  }
}
```

The Sliding Window Concept

- Communication is generally duplex; both sides transmit.
- **Piggybacking:** Instead of separate ACK frames, the acknowledgement is hooked onto the next outgoing data frame to save bandwidth.
- The sender maintains a *sending window* (sequence numbers it is permitted to send).
- The receiver maintains a *receiving window* (sequence numbers it is permitted to accept).
- Window size limits how many unacknowledged frames can be in transit.



A sliding window advancing at sender and receiver.

Protocol 4: One-Bit Sliding Window

Transfers data in both directions with stop-and-wait Piggybacks acks on reverse data frames for efficiency Handles transmission errors, flow control, early timers

```
void protocol4 (void) {
   seq_nr next_frame_to_send;
   seq_nr frame_expected;
   frame r, s;
   packet buffer;
   event_type event;
   next_frame_to_send = 0;
   frame_expected = 0;
   from_network_layer(&buffer);
   s.info = buffer;
   s.seq = next_frame_to_send;
   s.ack = 1 – frame_expected;
   to_physical_layer(&s);
   start_timer(s.seq);
```

Prepare first frame

Launch it, and set timer

. . .

Protocol 4: One-Bit Sliding Window

Wait for frame or timeout

If a frame with new data then deliver it

If an ack for last send then prepare for next data frame

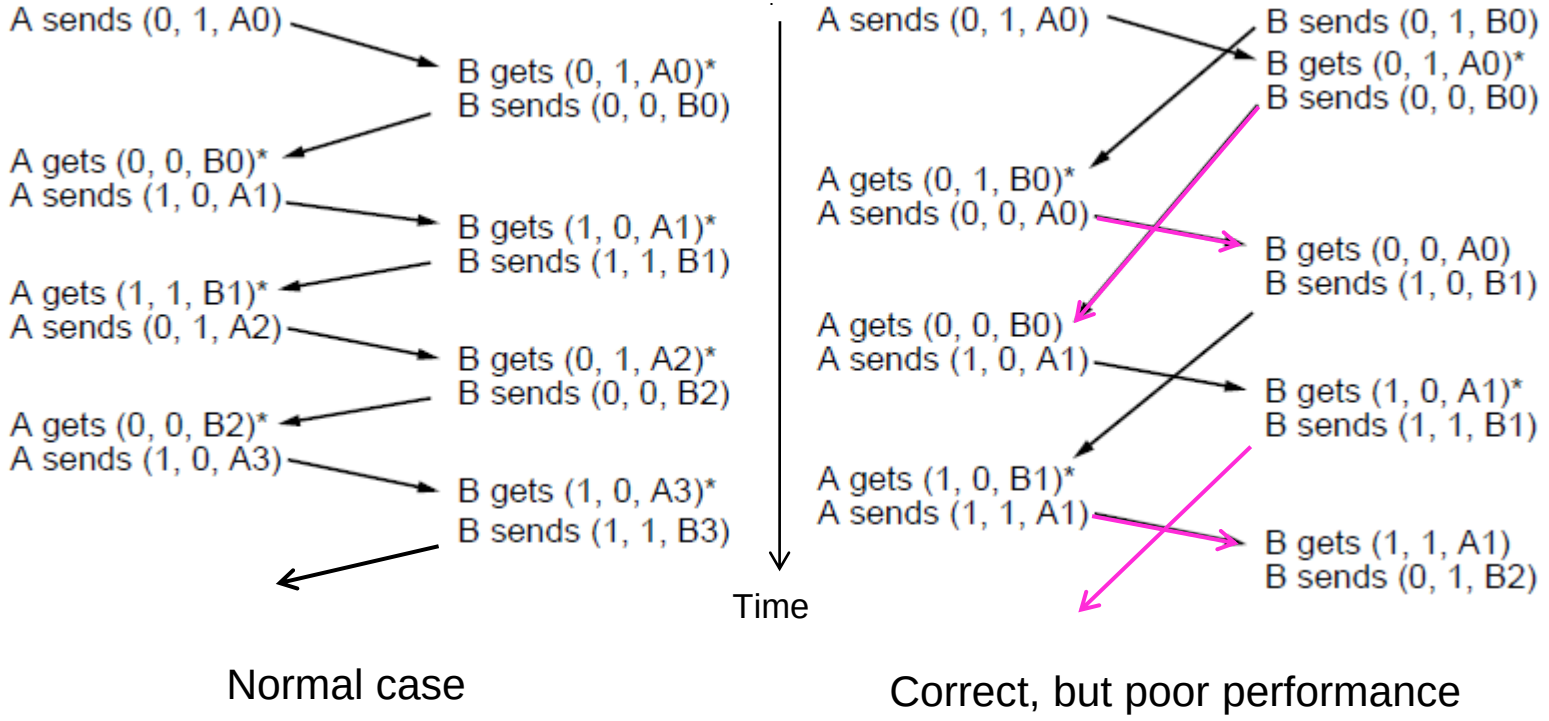
(Otherwise it was a timeout)

Send next data frame or retransmit old one; ack the last data we received

```
while (true) {
    wait_for_event(&event);
    if (event == frame_arrival) {
        from_physical_layer(&r);
        if (r.seq == frame_expected) {
            to_network_layer(&r.info);
            inc(frame_expected);
        }
        if (r.ack == next_frame_to_send) {
            stop_timer(r.ack);
            from_network_layer(&buffer);
            inc(next_frame_to_send);
        }
    }
    s.info = buffer;
    s.seq = next_frame_to_send;
    s.ack = 1 – frame_expected;
    to_physical_layer(&s);
    start_timer(s.seq);
  }
}
```

Protocol 4: One-Bit Sliding Window

- Transfers data in both directions using Stop-and-Wait semantics (Maximum window size = 1).
- Piggybacks ACKs on reverse data frames.
- Handles transmission errors and flow control.



Normal case

Correct, but poor performance

(a) Normal case. (b) Simultaneous Start Anomaly: Correct but slow performance due to duplicate frames.

The Need for Pipelining

Consider a 50-kbps satellite channel with a 500-msec round-trip delay, sending 1000-bit frames using stop-and-wait.

- Frame transmission takes 20 msec.
- Delay until ACK received is 500 msec.
- Total cycle time: 520 msec.
- **Efficiency:** Sender is active for 20/520 msec = ~4%. 96% of bandwidth is wasted!

Solution: Pipelining

- Permit the sender to transmit up to *w* frames before blocking for an acknowledgement.
- Keeps the pipe full. Appropriate value of *w* is related to the bandwidth-delay product. (2BD + 1)

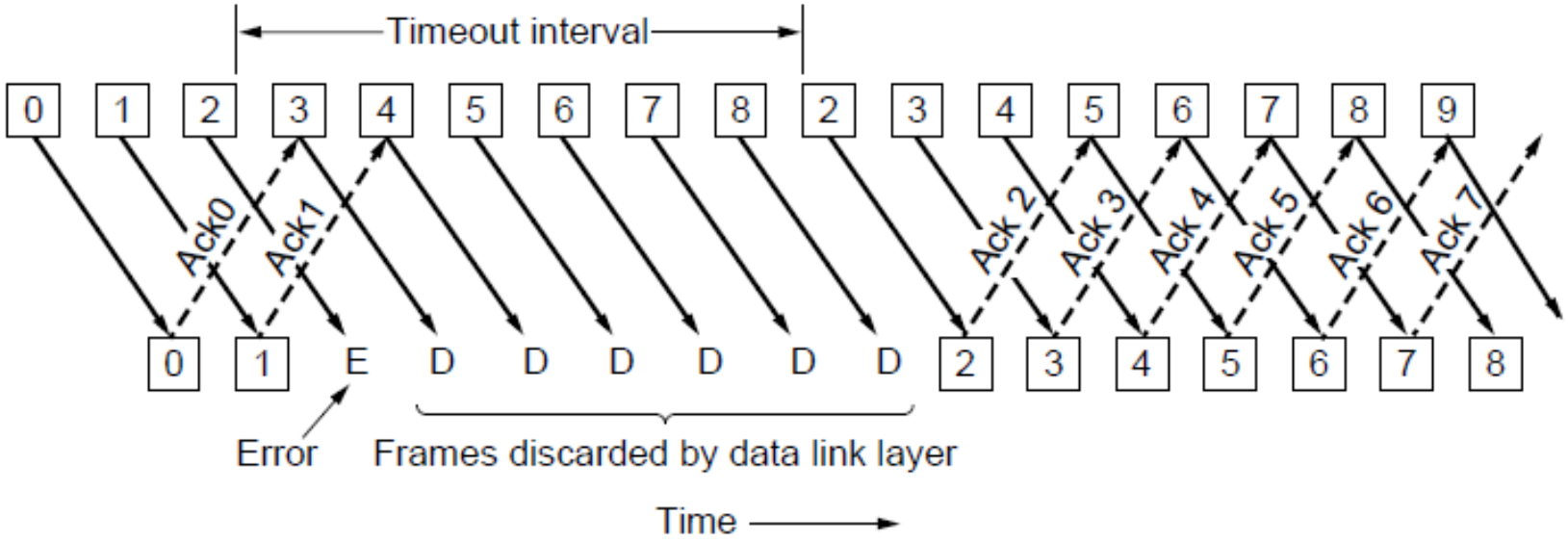
Larger windows enable pipelining for efficient link use

- Stop-and-wait (w=1) is inefficient for long links
- Best window (w) depends on bandwidth-delay (BD)
- Want w ≥ 2BD+1 to ensure high link utilization

Protocol 5: Go-Back-N

What happens when an error occurs in a pipelined transmission?

- In **Go-Back-N**, the receiver only accepts frames strictly in order (Receiver window size = 1).
- If frame *E* is corrupted, the receiver discards it.
- It also discards all subsequent valid frames (D, D, D) that arrive, sending no ACKs for them.
- The sender's timer for *E* expires, and it retransmits *E* and all subsequent frames in the window.
- **Tradeoff:** Simple receiver logic, but wastes bandwidth on errors.



Go-Back-N: Maximum Window Size

In Go-Back-N, the sender window size must be strictly less than the sequence number space.

If Sequence numbers are 0 to MAX_SEQ (space = MAX_SEQ + 1), Window Size must be MAX_SEQ.

Why? Example with Window = 8, Seq = 0-7:

- Sender sends frames 0 through 7.
- Receiver gets them all, sends ACK 7.
- ACK 7 is lost.
- Sender times out, retransmits frames 0-7.
- Receiver receives frame 0. Is this the *new* frame 0, or the *retransmitted* frame 0? Ambiguity!

By restricting the window size to 7 (MAX_SEQ), the receiver knows a received '0' must be a retransmission if it's expecting an '8' (or '0' of next batch).

Protocol 5: Go-Back-N

```
/* Protocol 5 (Go-back-n) allows multiple outstanding frames. The sender may transmit up
   to MAX_SEQ frames without waiting for an ack. In addition, unlike in the previous
   protocols, the network layer is not assumed to have a new packet all the time. Instead,
   the network layer causes a network_layer_ready event when there is a packet to send. */

#define MAX_SEQ 7
typedef enum {frame_arrival, cksum_err, timeout, network_layer_ready} event_type;
#include "protocol.h"

static boolean between(seq_nr a, seq_nr b, seq_nr c)
{
/* Return true if a <= b < c circularly; false otherwise. */
  if (((a <= b) && (b < c)) || ((c < a) && (a <= b)) || ((b < c) && (c < a)))
      return(true);
   else
      return(false);
}

static void send_data(seq_nr frame_nr, seq_nr frame_expected, packet buffer[])
{
/* Construct and send a data frame. */
  frame s;                                  /* scratch variable */

  s.info = buffer[frame_nr];                /* insert packet into frame */
  s.seq = frame_nr;                         /* insert sequence number into frame */
  s.ack = (frame_expected + MAX_SEQ) % (MAX_SEQ + 1);/* piggyback ack */
  to_physical_layer(&s);                    /* transmit the frame */
  start_timer(frame_nr);                    /* start the timer running */
}
```

Protocol 5: Go-Back-N

```
void protocol5(void)
{
  seq_nr next_frame_to_send;          /* MAX_SEQ > 1; used for outbound stream */
  seq_nr ack_expected;                /* oldest frame as yet unacknowledged */
  seq_nr frame_expected;              /* next frame expected on inbound stream */
  frame r;                            /* scratch variable */
  packet buffer[MAX_SEQ + 1];         /* buffers for the outbound stream */
  seq_nr nbuffered;                   /* number of output buffers currently in use */
  seq_nr i;                           /* used to index into the buffer array */
  event_type event;
  enable_network_layer();             /* allow network_layer_ready events */
  ack_expected = 0;                   /* next ack expected inbound */
  next_frame_to_send = 0;             /* next frame going out */
  frame_expected = 0;                 /* number of frame expected inbound */
  nbuffered = 0;                      /* initially no packets are buffered */

  while (true) {
    wait_for_event(&event);           /* four possibilities: see event_type above */
```

Protocol 5: Go-Back-N

```
switch(event) {
  case network_layer_ready:                  /* the network layer has a packet to send */
       /* Accept, save, and transmit a new frame. */
       from_network_layer(&buffer[next_frame_to_send]); /* fetch new packet */
       nbuffered = nbuffered + 1;            /* expand the sender's window */
       send_data(next_frame_to_send, frame_expected, buffer);/* transmit the frame */
       inc(next_frame_to_send);              /* advance sender's upper window edge */
       break;

  case frame_arrival:                        /* a data or control frame has arrived */
       from_physical_layer(&r);              /* get incoming frame from physical layer */

       if (r.seq == frame_expected) {
            /* Frames are accepted only in order. */
            to_network_layer(&r.info);       /* pass packet to network layer */
            inc(frame_expected);             /* advance lower edge of receiver's window */
        }
       /* Ack n implies n – 1, n – 2, etc.  Check for this. */
       while (between(ack_expected, r.ack, next_frame_to_send)) {
            /* Handle piggybacked ack. */
            nbuffered = nbuffered – 1;       /* one frame fewer buffered */
            stop_timer(ack_expected);        /* frame arrived intact; stop timer */
            inc(ack_expected);               /* contract sender's window */
       }
       break;
```

Protocol 5: Go-Back-N

```
      case cksum_err: break;                        /* just ignore bad frames */

      case timeout:                                 /* trouble; retransmit all outstanding frames */
            next_frame_to_send = ack_expected;      /* start retransmitting here */
            for (i = 1; i <= nbuffered; i++) {
                  send_data(next_frame_to_send, frame_expected, buffer);/* resend frame */
                  inc(next_frame_to_send);          /* prepare to send the next one */
            }

   }

    if (nbuffered < MAX_SEQ)
            enable_network_layer();
    else
            disable_network_layer();
  }
}
```

Protocol 6: Selective Repeat

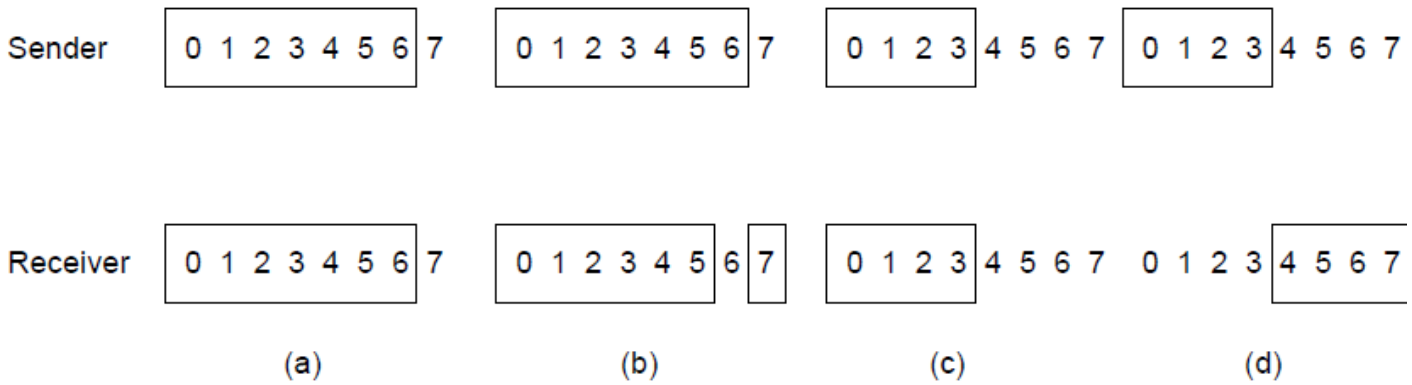
- Accepts frames out of order to save bandwidth (Receiver window size > 1).
- If a frame is corrupted, subsequent valid frames are buffered by the receiver.
- When a timer expires, **only that specific missing frame is retransmitted**.
- Uses Negative Acknowledgements (NAKs) to trigger fast retransmission before timeout.
- Requires significant buffer space at the receiver.

Selective Repeat: Window Size Limit

For Selective Repeat, the maximum window size is even more restricted to prevent sequence number overlap.

Crucial Rule: $$\text{Max Window Size} \leq \frac{(\text{MAX_SEQ} + 1)}{2}$$

- If the sequence space is 0-7 (8 numbers), the maximum window size is 4.
- This ensures that the new window and the old window do not overlap in sequence numbers.
- If the window size was 5, a delayed retransmission of frame 0 might be mistaken for the new frame 0 in the next window.



Initial situation with a window of size 7

a) After 7 frames sent and received

b) Acknowledgement is lost.

c) Initial situation with a window size of 4.

d) After 4 frames sent and received but not acknowledged.

Protocol 6: Selective Repeat

```
/* Protocol 6 (Selective repeat) accepts frames out of order but passes packets to the
   network layer in order. Associated with each outstanding frame is a timer. When the timer
   expires, only that frame is retransmitted, not all the outstanding frames, as in protocol 5. */

#define MAX_SEQ 7                                   /* should be 2^n – 1 */
#define NR_BUFS ((MAX_SEQ + 1)/2)
typedef enum {frame_arrival, cksum_err, timeout, network_layer_ready, ack_timeout} event_type;
#include "protocol.h"
boolean no_nak = true;                              /* no nak has been sent yet */
seq_nr oldest_frame = MAX_SEQ + 1;                  /* initial value is only for the simulator */

static boolean between(seq_nr a, seq_nr b, seq_nr c)
{
/* Same as between in protocol 5, but shorter and more obscure. */
  return ((a <= b) && (b < c)) || ((c < a) && (a <= b)) || ((b < c) && (c < a));
}
```

Protocol 6: Selective Repeat

```
static void send_frame(frame_kind fk, seq_nr frame_nr, seq_nr frame_expected, packet buffer[ ])
{
/* Construct and send a data, ack, or nak frame. */
  frame s;                                        /* scratch variable */

  s.kind = fk;                                    /* kind == data, ack, or nak */
  if (fk == data) s.info = buffer[frame_nr % NR_BUFS];
  s.seq = frame_nr;                               /* only meaningful for data frames */
  s.ack = (frame_expected + MAX_SEQ) % (MAX_SEQ + 1);
  if (fk == nak) no_nak = false;                  /* one nak per frame, please */
  to_physical_layer(&s);                          /* transmit the frame */
  if (fk == data) start_timer(frame_nr % NR_BUFS);
  stop_ack_timer();                               /* no need for separate ack frame */
}
void protocol6(void)
{
  seq_nr ack_expected;                            /* lower edge of sender's window */
  seq_nr next_frame_to_send;                      /* upper edge of sender's window + 1 */
  seq_nr frame_expected;                          /* lower edge of receiver's window */
  seq_nr too_far;                                 /* upper edge of receiver's window + 1 */
  int i;                                          /* index into buffer pool */
  frame r;                                        /* scratch variable */
  packet out_buf[NR_BUFS];                        /* buffers for the outbound stream */
  packet in_buf[NR_BUFS];                         /* buffers for the inbound stream */
  boolean arrived[NR_BUFS];                       /* inbound bit map */
  seq_nr nbuffered;                               /* how many output buffers currently used */
  event_type event;
```

Protocol 6: Selective Repeat

```
enable_network_layer();                          /* initialize */
ack_expected = 0;                                /* next ack expected on the inbound stream */
next_frame_to_send = 0;                          /* number of next outgoing frame */
frame_expected = 0;
too_far = NR_BUFS;
nbuffered = 0;                                   /* initially no packets are buffered */
for (i = 0; i < NR_BUFS; i++) arrived[i] = false;

while (true) {
  wait_for_event(&event);                        /* five possibilities: see event_type above */
  switch(event) {
    case network_layer_ready:                    /* accept, save, and transmit a new frame */
         nbuffered = nbuffered + 1;              /* expand the window */
         from_network_layer(&out_buf[next_frame_to_send % NR_BUFS]); /* fetch new packet */
         send_frame(data, next_frame_to_send, frame_expected, out_buf);/* transmit the frame */
         inc(next_frame_to_send);                /* advance upper window edge */
         break;
```

Protocol 6: Selective Repeat

```
case frame_arrival:                          /* a data or control frame has arrived */
    from_physical_layer(&r);                 /* fetch incoming frame from physical layer */
    if (r.kind == data) {
        /* An undamaged frame has arrived. */
        if ((r.seq != frame_expected) && no_nak)
            send_frame(nak, 0, frame_expected, out_buf); else start_ack_timer();
        if (between(frame_expected,r.seq,too_far) && (arrived[r.seq%NR_BUFS]==false)) {
            /* Frames may be accepted in any order. */
            arrived[r.seq % NR_BUFS] = true;      /* mark buffer as full */
            in_buf[r.seq % NR_BUFS] = r.info;     /* insert data into buffer */
            while (arrived[frame_expected % NR_BUFS]) {
                /* Pass frames and advance window. */
                to_network_layer(&in_buf[frame_expected % NR_BUFS]);
                no_nak = true;
                arrived[frame_expected % NR_BUFS] = false;
                inc(frame_expected);     /* advance lower edge of receiver's window */
                inc(too_far);            /* advance upper edge of receiver's window */
                start_ack_timer();       /* to see if a separate ack is needed */
            }
        }
    }
```

Protocol 6: Selective Repeat

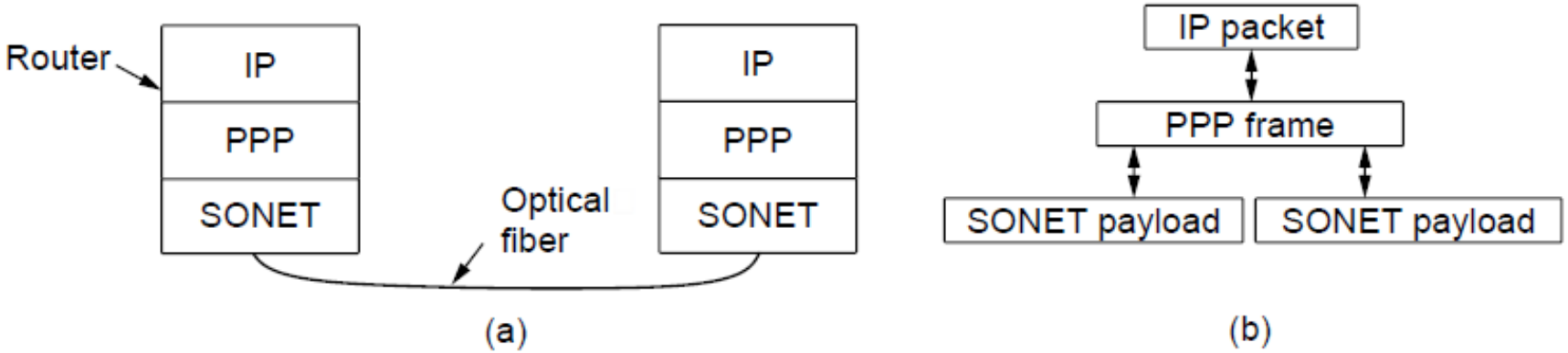
```
            if((r.kind==nak) && between(ack_expected,(r.ack+1)%(MAX_SEQ+1),next_frame_to_send))
                send_frame(data, (r.ack+1) % (MAX_SEQ + 1), frame_expected, out_buf);

            while (between(ack_expected, r.ack, next_frame_to_send)) {
                nbuffered = nbuffered – 1;              /* handle piggybacked ack */
                stop_timer(ack_expected % NR_BUFS);     /* frame arrived intact */
                inc(ack_expected);                      /* advance lower edge of sender's window */
            }
            break;
        case cksum_err:
            if (no_nak) send_frame(nak, 0, frame_expected, out_buf); /* damaged frame */
            break;
        case timeout:
            send_frame(data, oldest_frame, frame_expected, out_buf); /* we timed out */
            break;
        case ack_timeout:
            send_frame(ack,0,frame_expected, out_buf);      /* ack timer expired; send ack */
    }
    if (nbuffered < NR_BUFS) enable_network_layer(); else disable_network_layer();
  }
}
```

Example Protocols: Packet over SONET

SONET Backbone

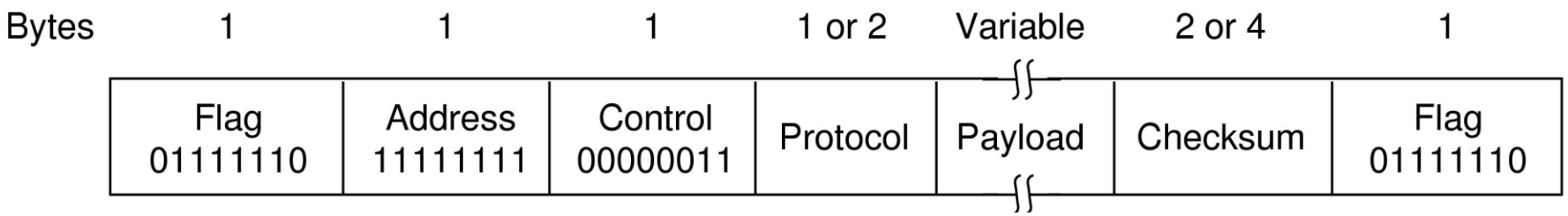
- SONET is a physical layer protocol used over wide-area optical fiber links (the Internet backbone).
- It provides a synchronous bitstream at a well-defined rate.
- Routers use the Point-to-Point Protocol (PPP) as the data link layer framing mechanism over SONET links.



(a) Protocol Stack (b) Frame relationships

Point-to-Point Protocol (PPP) Features

- **Framing and Error Detection:** Uses a byte-oriented frame format.
- **Link Control Protocol (LCP):** Used for bringing lines up, testing them, negotiating options, and bringing them down gracefully.
- **Network Control Protocol (NCP):** Used to negotiate network-layer options independently of the underlying protocol.

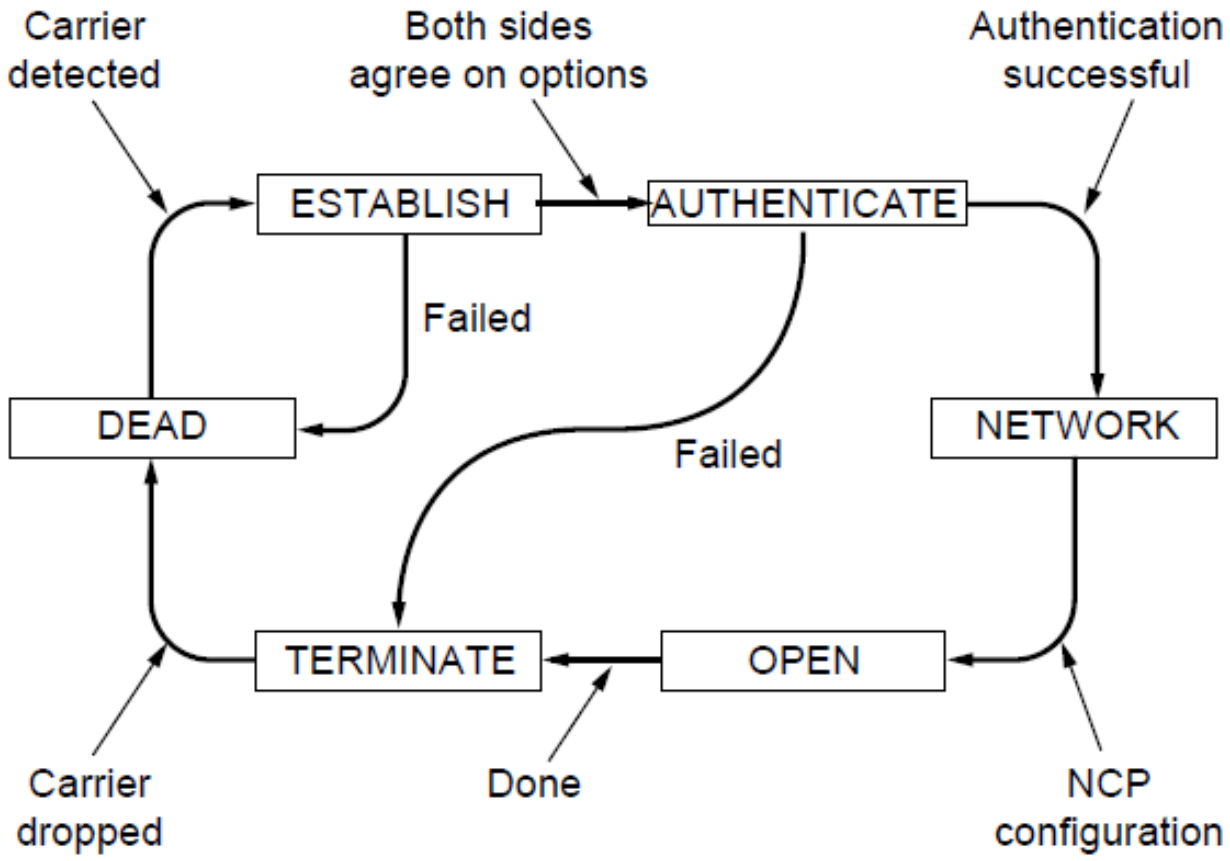


PPP frame format closely resembles HDLC, but is byte-oriented. Uses 0x7E as flag and byte stuffing.

- Flag byte 01111110 (0X7E), Esc byte 0X7D. If 0X7E is encountered in data it is stuffed with 0X7D and next byte is XORed with 0x20. So 0X7E is sent as 0X7D 0X5E.
- Point to point link does not need address. So it is set to 11111111
- Control indicates type of frame - here it indicates unnumbered frame
- Protocol indicates type of payload carried

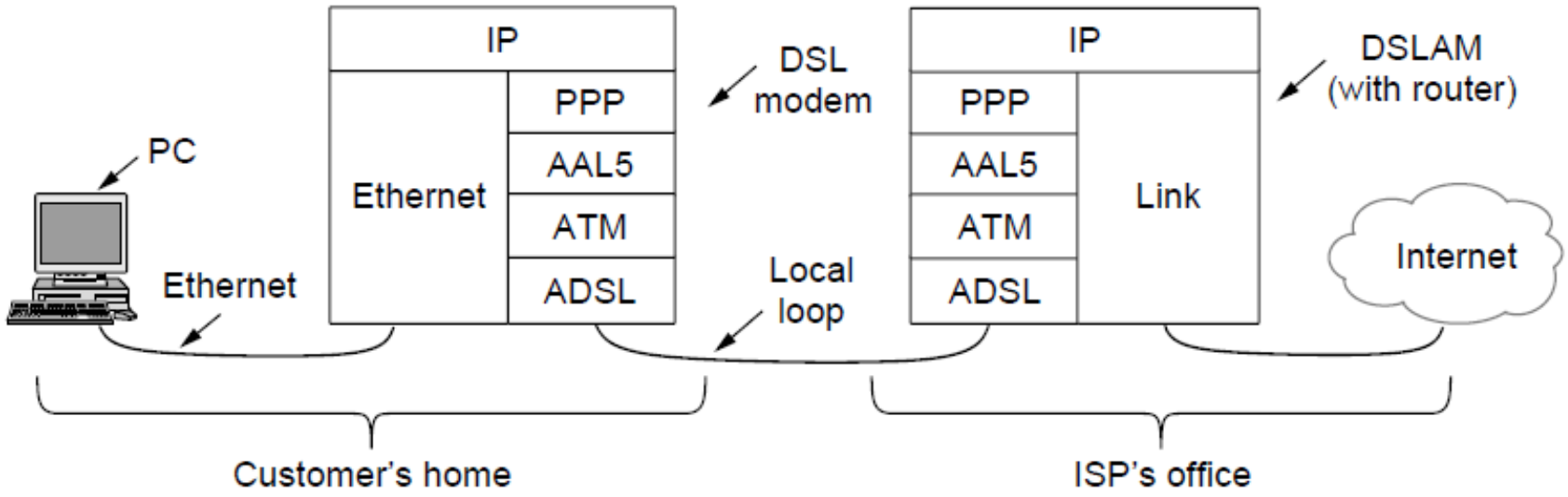
PPP Link State Diagram

The lifecycle of a PPP connection managed by LCP.



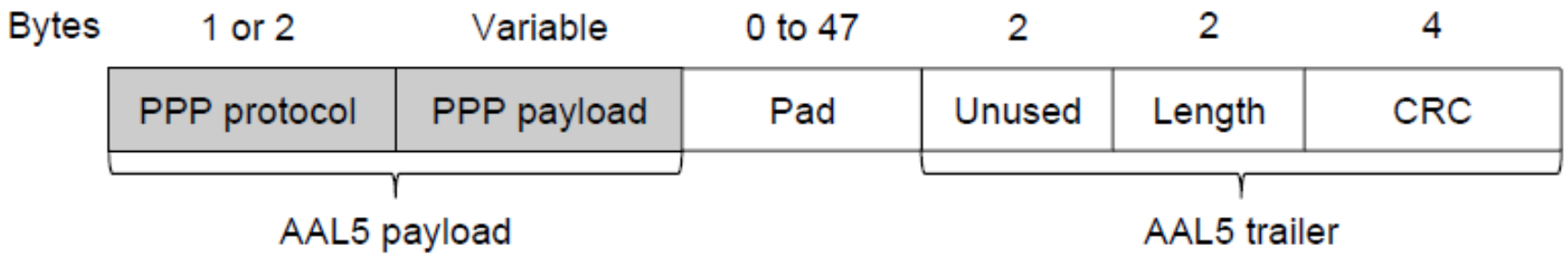
Example Protocols: ADSL Architecture

- Asymmetric Digital Subscriber Loop provides broadband over legacy copper telephone lines.
- Because it was built over existing infrastructure, the protocol stack is famously complex.
- A typical data path: PC -> Ethernet -> DSL Modem -> PPPoE -> AAL5 -> ATM -> ADSL -> DSLAM at the ISP.



AAL5 Frame Format (Used in ADSL)

In the ADSL stack, the PPP frame is encapsulated inside an AAL5 frame before being sliced into ATM cells.



AAL5 payload contains the PPP data, padded to a multiple of 48 bytes, followed by a trailer with length and CRC.

End of Chapter 3

The Data Link Layer